

video_detector.py 技術報告

逐行中文說明與 Debug 地圖

這份 PDF 說明 AutoCamTracker V1 第一個模組 video_detector.py。內容包含模組責任、資料流、重要資料結構、常見 debug 點，以及每一行程式碼的中文功能說明。英文術語已盡量轉成中文；必要技術名詞保留原字並附中
文解釋。

- 模組定位：輸入影片與 YOLO 影像辨識
- 輸入來源：MacBook 鏡頭、本機影片檔、螢幕區域擷取
- 追蹤方式：Ultralytics YOLO track 模式，支援 BoT-SORT 與 Deep OC-SORT
- 輸出資料：原始畫面與 TrackedDetection 追蹤偵測資料
- 不負責：目標選取、UI 排版、重新構圖、錄影寫檔

模組總覽

模組責任

- 開啟 MacBook 鏡頭、本機影片檔或螢幕區域擷取來源。
- 載入 Ultralytics YOLO 模型。
- 使用 YOLO 追蹤模式，搭配 BoT-SORT 或 Deep OC-SORT。
- 輸出原始畫面與追蹤偵測資料。

不負責範圍

- 不負責使用者選取哪台車。
- 不負責單選、多選、目標遺失重置。
- 不負責重新構圖、數位變焦或畫面裁切。
- 不負責 Tkinter 介面排版與錄影檔案寫入。

資料流

```
InputConfig 設定
-> VideoDetector.open_source() 開啟來源
-> VideoDetector.read_frame() 讀取單張畫面
-> VideoDetector.track_frame() 執行 YOLO 追蹤
-> VideoDetector._parse_results() 解析結果
-> list[TrackedDetection] 追蹤偵測資料
```

Debug 優先檢查點

- 鏡頭打不開：檢查 `open_source()` 第 87-89 行。
- 模型載入失敗：檢查 `load_model()` 第 69-72 行。
- 有畫面但沒有框：檢查 `_parse_results()` 第 162-164 行。
- 有偵測但被過濾：檢查車輛類別與信心分數，第 176-179 行。
- 後續無法追蹤：檢查 `track_id` 是否為空，第 182 行。

InputConfig：輸入與模型設定

- source_type：輸入來源，包含鏡頭、影片檔、螢幕區域。
- camera_index：攝影機編號，MacBook 內建鏡頭通常是 0。
- video_path：本機影片檔路徑。
- screen_region：螢幕擷取區域，格式是 x、y、寬、高。
- model_path：YOLO 模型檔案路徑。
- tracker_name：追蹤器名稱，BoT-SORT 或 Deep OC-SORT。
- confidence_threshold：信心分數門檻。
- iou_threshold：交集聯集比門檻。
- vehicle_classes_only：是否只保留車輛類別。

TrackedDetection：追蹤偵測輸出

- track_id：追蹤 ID，同一台車跨畫面幀的身份。
- bbox：邊界框座標。
- class_id / class_name：類別編號與類別名稱。
- confidence：信心分數。
- center：中心點。
- frame_index：畫面幀編號。
- timestamp：時間戳記。
- tracker_name：使用的追蹤器名稱。

逐行說明

1	"""Input Video + YOLO Detection module for AutoCamTracker V1.	開始檔案說明文字，標明這是輸入影片與 YOLO 偵測模組。
2	空行	空行，讓說明文字分段。
3	Responsibilities:	說明接下來列出這個模組的責任。
4	- Open webcam, local video file, or screen-region sources.	責任一：開啟網路攝影機、本機影片檔或螢幕區域來源。
5	- Load an Ultralytics YOLO model.	責任二：載入 Ultralytics YOLO 模型。
6	- Run YOLO tracking with BoT-SORT or Deep OC-SORT.	責任三：使用 BoT-SORT 或 Deep OC-SORT 做 YOLO 追蹤。
7	- Return raw frames plus tracked detections.	責任四：回傳原始畫面與追蹤偵測結果。
8	空行	空行，分隔責任列表與邊界說明。
9	This module should not manage target selection, UI layout, reframeing, or	說明這個模組不管理目標選取、使用者介面、重新構圖等功能。
10	recording file output.	延續上一行，說明也不負責錄影檔案輸出。
11	"""	結束檔案說明文字。
12	空行	空行或結構分隔，用來提升程式可讀性。
13	from __future__ import annotations	啟用延後型別註解解析，避免型別在執行時太早被解析。
14	空行	空行或結構分隔，用來提升程式可讀性。
15	from dataclasses import dataclass	匯入 dataclass，用來快速建立設定與資料物件。
16	from time import time	匯入目前時間函式，用來給偵測結果加時間戳記。
17	from typing import Any, Iterable, Literal	匯入型別工具：任意型別、可迭代資料、固定字串型別。
18	空行	空行或結構分隔，用來提升程式可讀性。
19	空行	空行或結構分隔，用來提升程式可讀性。
20	SourceType = Literal["webcam", "video_file", "screen_region"]	定義輸入來源型別，只允許鏡頭、影片檔、螢幕區域三種。
21	TrackerName = Literal["botsort", "deepocsort"]	定義追蹤器名稱型別，只允許 BoT-SORT 與 Deep OC-SORT。
22	空行	空行或結構分隔，用來提升程式可讀性。
23	空行	空行或結構分隔，用來提升程式可讀性。
24	TRACKER_CONFIGS: dict[TrackerName, str] = {	建立追蹤器設定對照表，key 是程式內名稱，value 是 Ultralytics 設定檔。
25	"botsort": "botsort.yaml",	BoT-SORT 對應 Ultralytics 的 botsort.yaml。
26	"deepocsort": "deepocsort.yaml",	Deep OC-SORT 對應 Ultralytics 的 deepocsort.yaml。

逐行說明

27	}	結束追蹤器設定對照表。
28	空行	空行或結構分隔，用來提升程式可讀性。
29	VEHICLE_CLASS_NAMES = {"car", "truck", "bus", "motorcycle"}	定義 V1 要保留的車輛類別名稱集合。
30	空行	空行或結構分隔，用來提升程式可讀性。
31	空行	空行或結構分隔，用來提升程式可讀性。
32	@dataclass	宣告下一個 class 是資料類別，Python 會自動產生初始化方法。
33	class InputConfig:	定義輸入與模型設定物件。
34	source_type: SourceType = "webcam"	設定輸入來源，預設為網路攝影機。
35	camera_index: int = 0	設定攝影機編號，MacBook 內建鏡頭通常是 0。
36	video_path: str None = None	設定本機影片路徑，只有影片檔模式需要。
37	screen_region: tuple[int, int, int, int] None = None	設定螢幕擷取區域，格式是 x、y、寬、高。
38	model_path: str = "yolo11n.pt"	設定 YOLO 模型路徑，預設使用 yolo11n.pt。
39	tracker_name: TrackerName = "botsort"	設定追蹤器，預設使用 BoT-SORT。
40	confidence_threshold: float = 0.25	設定信心分數門檻，低於此數值的偵測結果會被過濾。
41	iou_threshold: float = 0.7	設定 IoU 門檻，用於 YOLO 推論與追蹤匹配。
42	vehicle_classes_only: bool = True	設定是否只保留車輛類別。
43	空行	空行或結構分隔，用來提升程式可讀性。
44	空行	空行或結構分隔，用來提升程式可讀性。
45	@dataclass	宣告下一個 class 是資料類別。
46	class TrackedDetection:	定義單一追蹤偵測結果的資料格式。
47	track_id: int None	追蹤 ID，代表同一台車跨幀的身份；可能為空。
48	bbox: tuple[float, float, float, float]	邊界框座標，格式是左上與右下座標。
49	class_id: int	YOLO 類別編號。
50	class_name: str	YOLO 類別名稱，例如 car 或 truck。
51	confidence: float	偵測信心分數。
52	center: tuple[float, float]	邊界框中心點座標。
53	frame_index: int	這筆資料來自第幾幀。

逐行說明

54	<code>timestamp: float</code>	這筆資料建立時的時間戳記。
55	<code>tracker_name: TrackerName</code>	這筆資料使用的追蹤器名稱。
56	空行	空行或結構分隔，用來提升程式可讀性。
57	空行	空行或結構分隔，用來提升程式可讀性。
58	<code>class VideoDetector:</code>	定義 VideoDetector 類別，負責讀取畫面與執行 YOLO 追蹤。
59	<code>"""Reads frames and runs Ultralytics YOLO track mode."""</code>	類別說明文字：讀取畫面並執行 YOLO 追蹤模式。
60	空行	空行或結構分隔，用來提升程式可讀性。
61	<code>def __init__(self, config: InputConfig) -> None:</code>	初始化 VideoDetector，需要傳入 InputConfig。
62	<code>self.config = config</code>	保存設定物件，後續所有方法都會讀取它。
63	<code>self.model: Any None = None</code>	YOLO 模型物件，初始化時尚未載入。
64	<code>self.capture: Any None = None</code>	OpenCV 影像來源物件，初始化時尚未開啟。
65	<code>self.screen_capture: Any None = None</code>	螢幕擷取物件，初始化時尚未開啟。
66	<code>self.frame_index = 0</code>	目前讀到第幾幀，從 0 開始。
67	<code>self_cv2 = None</code>	保留 OpenCV 模組參照，目前主要是除錯與延伸用途。
68	空行	空行或結構分隔，用來提升程式可讀性。
69	<code>def load_model(self) -> None:</code>	定義載入模型的方法。
70	<code>from ultralytics import YOLO</code>	在方法內匯入 YOLO，避免只 import 檔案時就需要 ultralytics 套件。
71	空行	空行或結構分隔，用來提升程式可讀性。
72	<code>self.model = YOLO(self.config.model_path)</code>	用設定中的模型路徑載入 YOLO 模型。
73	空行	空行或結構分隔，用來提升程式可讀性。
74	<code>def open_source(self) -> None:</code>	定義開啟輸入來源的方法。
75	<code>if self.config.source_type in {"webcam", "video_file"}:</code>	如果來源是攝影機或影片檔，就走 OpenCV 流程。
76	<code>import cv2</code>	匯入 OpenCV。
77	空行	空行或結構分隔，用來提升程式可讀性。
78	<code>self_cv2 = cv2</code>	保存 OpenCV 模組參照。
79	<code>source: int str</code>	宣告 source 變數可能是整數或字串。
80	<code>if self.config.source_type == "webcam":</code>	判斷是否為攝影機模式。
81	<code>source = self.config.camera_index</code>	攝影機模式使用 camera_index 作為來源。

逐行說明

82	else:	否則就是影片檔模式。
83	if not self.config.video_path:	影片檔模式檢查是否有影片路徑。
84	raise ValueError("video_path is required for video _file input")	沒有影片路徑就丟出錯誤。
85	source = self.config.video_path	影片檔模式使用 video_path 作為來源。
86	空行	空行或結構分隔，用來提升程式可讀性。
87	self.capture = cv2.VideoCapture(source)	用 OpenCV 開啟攝影機或影片檔。
88	if not self.capture.isOpened():	檢查來源是否成功開啟。
89	raise RuntimeError(f"Unable to open input source: { source}")	開啟失敗就丟出執行時錯誤。
90	空行	空行或結構分隔，用來提升程式可讀性。
91	elif self.config.source_type == "screen_region":	如果來源是螢幕區域，就走螢幕擷取流程。
92	if self.config.screen_region is None:	檢查是否有指定螢幕區域。
93	raise ValueError("screen_region is required for scre en_region input")	沒有螢幕區域就丟出錯誤。
94	import mss	匯入 mss 螢幕擷取套件。
95	空行	空行或結構分隔，用來提升程式可讀性。
96	self.screen_capture = mss.mss()	建立 mss 螢幕擷取物件。
97	空行	空行或結構分隔，用來提升程式可讀性。
98	else:	如果來源類型不在支援範圍內。
99	raise ValueError(f"Unsupported source_type: {self.con fig.source_type}")	丟出不支援來源類型的錯誤。
100	空行	空行或結構分隔，用來提升程式可讀性。
101	def read_frame(self) -> Any None:	定義讀取單張畫面的方法。
102	if self.config.source_type in {"webcam", "video_file"}:	攝影機與影片檔都用 OpenCV 讀取。
103	if self.capture is None:	確認影像來源已經開啟。
104	raise RuntimeError("Input source is not open")	未開啟就丟出錯誤。
105	ok, frame = self.capture.read()	從 OpenCV 來源讀取一幀，回傳成功旗標與畫面。
106	if not ok:	如果讀取失敗。
107	return None	回傳空值，代表影片結束或來源中斷。

逐行說明

108	<code>self.frame_index += 1</code>	成功讀取後，幀編號加一。
109	<code>return frame</code>	回傳原始畫面。
110	空行	空行或結構分隔，用來提升程式可讀性。
111	<code>if self.config.source_type == "screen_region":</code>	螢幕區域模式使用另一套讀取流程。
112	<code>if self.screen_capture is None:</code>	確認螢幕擷取物件已經建立。
113	<code>raise RuntimeError("Screen capture source is not open")</code>	未建立就丟出錯誤。
114	<code>import cv2</code>	匯入 OpenCV，用於色彩格式轉換。
115	<code>import numpy as np</code>	匯入 NumPy，用於把螢幕截圖轉成陣列。
116	空行	空行或結構分隔，用來提升程式可讀性。
117	<code>x, y, width, height = self.config.screen_region or (0, 0, 0, 0)</code>	讀取螢幕區域座標；若沒有則使用零區域保底。
118	<code>image = self.screen_capture.grab(</code>	呼叫 mss 擷取螢幕畫面。
119	<code>{ "left": x, "top": y, "width": width, "height": height }</code>	傳入左上角座標與寬高。
120	<code>)</code>	結束 grab 參數。
121	<code>frame = np.array(image)</code>	把擷取結果轉成 NumPy 陣列。
122	<code>frame = cv2.cvtColor(frame, cv2.COLOR_BGRA2BGR)</code> <code>)</code>	把 BGRA 色彩格式轉為 OpenCV 常用的 BGR。
123	<code>self.frame_index += 1</code>	成功讀取後，幀編號加一。
124	<code>return frame</code>	回傳螢幕擷取畫面。
125	空行	空行或結構分隔，用來提升程式可讀性。
126	<code>return None</code>	如果來源類型沒有匹配任何流程，回傳空值。
127	空行	空行或結構分隔，用來提升程式可讀性。
128	<code>def track_frame(self, frame: Any) -> list[TrackedDetection]:</code>	定義對單張畫面執行追蹤的方法。
129	<code>if self.model is None:</code>	檢查 YOLO 模型是否已載入。
130	<code>raise RuntimeError("YOLO model is not loaded")</code>	未載入模型就丟出錯誤。
131	空行	空行或結構分隔，用來提升程式可讀性。
132	<code>tracker_config = TRACKER_CONFIGS[self.config.tracker_name]</code>	根據設定選擇追蹤器 yaml 檔名。
133	<code>results = self.model.track(</code>	呼叫 YOLO 的追蹤模式。

逐行說明

134	frame,	傳入當前畫面。
135	persist=True,	啟用跨幀保留狀態，這是取得穩定 track_id 的關鍵。
136	tracker=tracker_config,	指定追蹤器設定檔。
137	conf=self.config.confidence_threshold,	傳入信心分數門檻。
138	iou=self.config.iou_threshold,	傳入 IoU 門檻。
139	verbose=False,	關閉 Ultralytics 詳細輸出。
140	)	結束 model.track 呼叫。
141	return self._parse_results(results)	把 YOLO 原始結果解析成 TrackedDetection 清單。
142	空行	空行或結構分隔，用來提升程式可讀性。
143	def read_and_track(self) -> tuple[Any None, list[TrackedDetection]]:	定義一次完成讀取畫面與追蹤的方法。
144	frame = self.read_frame()	先讀取一張畫面。
145	if frame is None:	如果沒有讀到畫面。
146	return None, []	回傳空畫面與空偵測結果。
147	detections = self.track_frame(frame)	對畫面執行追蹤。
148	return frame, detections	回傳原始畫面與追蹤偵測結果。
149	空行	空行或結構分隔，用來提升程式可讀性。
150	def close(self) -> None:	定義關閉資源的方法。
151	if self.capture is not None:	如果 OpenCV 來源存在。
152	self.capture.release()	釋放 OpenCV 影像來源。
153	if self.screen_capture is not None:	如果螢幕擷取物件存在。
154	self.screen_capture.close()	關閉螢幕擷取物件。
155	空行	空行或結構分隔，用來提升程式可讀性。
156	def _parse_results(self, results: Iterable[Any]) -> list[TrackedDetection]:	定義解析 YOLO 結果的內部方法。
157	parsed: list[TrackedDetection] = []	建立解析後結果清單。
158	timestamp = time()	建立目前時間戳記，給本批結果共用。
159	空行	空行或結構分隔，用來提升程式可讀性。
160	for result in results:	逐一處理 YOLO 回傳的 result。

逐行說明

161	<code>names = getattr(result, "names", {}) or {}</code>	取得類別編號到名稱的對照表。
162	<code>boxes = getattr(result, "boxes", None)</code>	取得 boxes 物件，裡面包含 bbox、類別、信心分數、追蹤 ID。
163	<code>if boxes is None:</code>	如果沒有 boxes。
164	<code>continue</code>	跳過這個 result。
165	空行	空行或結構分隔，用來提升程式可讀性。
166	<code>xyxy = self._to_list(getattr(boxes, "xyxy", []))</code>	取得 bbox 座標並轉成 Python list。
167	<code>cls_values = self._to_list(getattr(boxes, "cls", []))</code>	取得類別編號並轉成 Python list。
168	<code>conf_values = self._to_list(getattr(boxes, "conf", []))</code>	取得信心分數並轉成 Python list。
169	<code>id_values = self._to_list(getattr(boxes, "id", []))</code>	取得追蹤 ID 並轉成 Python list。
170	空行	空行或結構分隔，用來提升程式可讀性。
171	<code>for index, bbox_values in enumerate(xyxy):</code>	逐一處理每個 bbox。
172	<code>class_id = int(cls_values[index]) if index < len(cls_values) else -1</code>	取得目前 bbox 的類別編號；若資料不足就用 -1。
173	<code>class_name = str(names.get(class_id, class_id))</code>	用類別編號查類別名稱；查不到就用編號字串。
174	<code>confidence = float(conf_values[index]) if index < len(conf_values) else 0.0</code>	取得信心分數；若資料不足就用 0。
175	空行	空行或結構分隔，用來提升程式可讀性。
176	<code>if self.config.vehicle_classes_only and class_name not in VEHICLE_CLASS_NAMES:</code>	如果設定只保留車輛，且類別不在車輛清單內。
177	<code>continue</code>	跳過非車輛目標。
178	<code>if confidence < self.config.confidence_threshold:</code>	如果信心分數低於門檻。
179	<code>continue</code>	跳過低信心分數目標。
180	空行	空行或結構分隔，用來提升程式可讀性。
181	<code>x1, y1, x2, y2 = [float(value) for value in bbox_values]</code>	將 bbox 四個座標轉為浮點數。
182	<code>track_id = int(id_values[index]) if index < len(id_values) else None</code>	取得追蹤 ID；如果沒有 ID，設定為空值。
183	<code>parsed.append(</code>	把解析好的資料加入結果清單。
184	<code>TrackedDetection(</code>	建立 TrackedDetection 物件。
185	<code>track_id=track_id,</code>	寫入追蹤 ID。
186	<code>bbox=(x1, y1, x2, y2),</code>	寫入邊界框。

逐行說明

187	<code>class_id=class_id,</code>	寫入類別編號。
188	<code>class_name=class_name,</code>	寫入類別名稱。
189	<code>confidence=confidence,</code>	寫入信心分數。
190	<code>center=((x1 + x2) / 2.0, (y1 + y2) / 2.0),</code>	計算並寫入中心點。
191	<code>frame_index=self.frame_index,</code>	寫入目前幀編號。
192	<code>timestamp=timestamp,</code>	寫入時間戳記。
193	<code>tracker_name=self.config.tracker_name,</code>	寫入追蹤器名稱。
194	<code>)</code>	結束 TrackedDetection 建立。
195	<code>)</code>	結束 append 呼叫。
196	空行	空行或結構分隔，用來提升程式可讀性。
197	<code>return parsed</code>	回傳所有解析後的追蹤偵測結果。
198	空行	空行或結構分隔，用來提升程式可讀性。
199	<code>@staticmethod</code>	宣告下一個方法是靜態方法，不需要使用 self。
200	<code>def _to_list(value: Any) -> list[Any]:</code>	定義轉換成 Python list 的工具方法。
201	<code>if value is None:</code>	如果傳入值是空值。
202	<code>return []</code>	回傳空清單。
203	<code>if hasattr(value, "cpu"):</code>	如果物件有 cpu 方法，通常代表是 PyTorch 張量。
204	<code>value = value.cpu()</code>	先把張量移到 CPU。
205	<code>if hasattr(value, "numpy"):</code>	如果物件有 numpy 方法。
206	<code>value = value.numpy()</code>	轉成 NumPy 陣列。
207	<code>if hasattr(value, "tolist"):</code>	如果物件有 tolist 方法。
208	<code>return value.tolist()</code>	轉成 Python list。
209	<code>return list(value)</code>	最後備援：用 list 建構子轉成清單。